

WEEK-7: CNN Implementation on MNIST Dataset

Step 1: Install Required Libraries

```
pip install tensorflow
```

Step 2: Import Libraries

```
import tensorflow as tf  
from tensorflow import keras  
from tensorflow.keras import layers  
import numpy as np  
import matplotlib.pyplot as plt
```

Step 3: Load the MNISTDataset

```
# Load the dataset  
mnist = keras.datasets.mnist  
  
# Split the dataset into training and testing data  
(x_train, y_train), (x_test, y_test) = mnist.load_data()  
  
# Normalize the images to values between 0 and 1  
x_train = x_train.astype('float32') / 255  
x_test = x_test.astype('float32') / 255  
  
# Reshape the images to add the channel dimension (28x28x1)  
x_train = np.expand_dims(x_train,-1)  
x_test = np.expand_dims(x_test,-1)
```

Step 4: Build the CNN Model

```
model = keras.Sequential([  
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu',  
        input_shape=(28, 28, 1)),
```

```
layers.MaxPooling2D(pool_size=(2, 2)),
layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
layers.MaxPooling2D(pool_size=(2, 2)),
layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dense(10, activation='softmax')
])

# Print the model summary
model.summary()
```

Step 5: Compile the Model

```
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Step 6: Train the Model

```
model.fit(x_train, y_train, epochs=5, batch_size=64,
validation_split=0.2)
```

Step 7: Evaluate the Model

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f'Test accuracy: {test_accuracy:.4f}')
```

Step 8: Make Predictions

```
predictions = model.predict(x_test)

# Display the first test image and its predicted label
plt.imshow(x_test[0].reshape(28, 28), cmap='gray')
plt.title(f'Predicted label: {np.argmax(predictions[0])}')
plt.show()
```